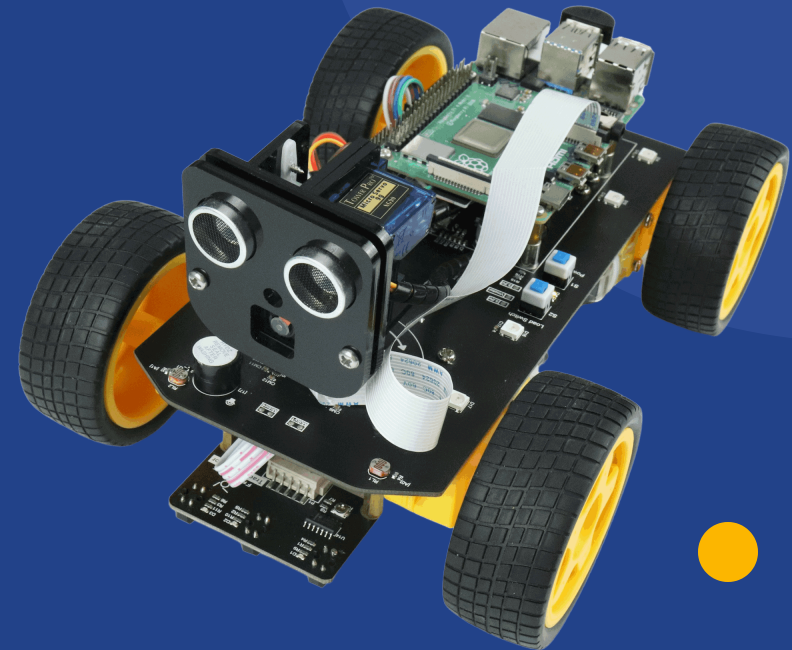




LESSON 4 – AUTONOMOUS MODES

Machine Learning with Raspberry Pi & Smart Car

Today the car makes its own decisions – using its sensors to drive itself, with no commands from us.



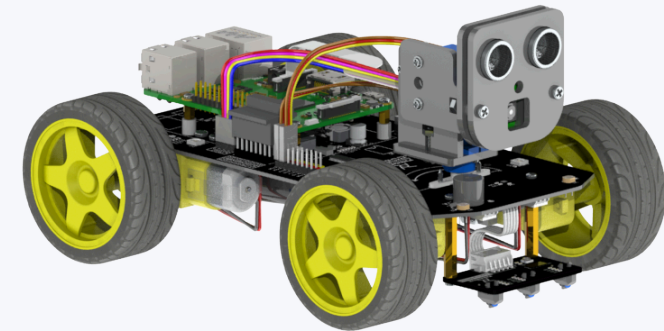


LESSON 4 – AUTONOMOUS MODES

From remote control to autonomy

So far we told the car exactly what to do. Today it decides for itself.

- You've used **motors** (L2), **light + battery sensors** (L3), the **servo head** and **ultrasonic** (L1).
- Until now, every move came from a **command we sent**.
- **Today:** the car uses those same sensors to **drive itself** – no commands from us.





LESSON 4 – AUTONOMOUS MODES

Today's goal: one car, two brains

Build a self-driving car with two modes – pick one with `DEMO_MODE`.

"light" **M-Light**

Drive toward the **brightest light** using the two light sensors.

"sonic" **M-Sonic**

Look around with the head + ultrasonic and **steer around obstacles**.



Both modes run the same idea: the **Sense** → **Think** → **Act** loop that powers all robots.



LESSON 4 – AUTONOMOUS MODES

Sense → Think → Act

Every autonomous robot repeats three steps, fast – many times a second.

1

Sense

Read the sensors – light voltages, or distance ahead.

2

Think

Decide what to do with those numbers – go, turn, or stop.

3

Act

Drive the motors based on the decision.



Our `loop()` repeats these three steps over and over – that's the whole lesson.



LESSON 4 – AUTONOMOUS MODES • THE FILE, CHUNK 1

Start the file: imports

Pull in motors plus the sensors both modes might use.

- `import car_setup` first – it finds the car's modules.
- The **motor** drives; both modes need it.
- `from servo import Servo` – remember, the file is lowercase `servo.py`.

```
lesson_4_autonomous_modes.py

"""Lesson 4: Autonomous Modes."""

import car_setup      # add the car's code folders
import time

from motor import Motor      # driving wheels
from adc import Adc          # light sensors
from servo import Servo      # turns the head
from ultrasonic import Ultrasonic  # measures distance
```



LESSON 4 – AUTONOMOUS MODES • THE FILE, CHUNK 2

Settings: speed and mode

`DEMO_MODE` chooses which brain runs; two speeds control how it drives.

- `DEMO_MODE` picks the brain – `"light"` or `"sonic"`.
- `DRIVE_SPEED` = a gentle forward power.
- `TURN_SPEED` = a stronger turning power. Keep both modest indoors.

```
lesson_4_autonomous_modes.py

# ---- settings you can change ----
DEMO_MODE = "sonic"      # "light" or "sonic"
DRIVE_SPEED = 1200       # forward power (0..4095)
TURN_SPEED = 1500        # turning power
```



LESSON 4 – AUTONOMOUS MODES • THE FILE, CHUNK 3

M-Light: follow the light

Two light sensors on ADC channels 0 and 1 – **lower voltage means more light**.



Both sides bright → **go straight**



One side brighter → **turn toward it**



Dim and even → **wait**

```
lesson_4_autonomous_modes.py

# M-Light: light sensors on channels 0, 1.
# LOWER voltage = MORE light.
LEFT_LIGHT = 0
RIGHT_LIGHT = 1
BRIGHT_ENOUGH = 2.99      # below this = light
SIDE_DIFFERENCE = 0.15    # gap to turn
```

LESSON 4 – AUTONOMOUS MODES • THE FILE, CHUNK 4

M-Sonic: dodge obstacles

Use the head servo + ultrasonic to look **ahead, left, and right**.

- Three head angles to check: **left, center, right**.
- Anything closer than `BLOCKED` cm counts as an **obstacle**.
- Same `setServoPwm("0", angle)` head-sweep idea as Lesson 1.



ULTRASONIC DISTANCE SENSOR

```
lesson_4_autonomous_modes.py

# M-Sonic: head angles, and "too close".
HEAD_LEFT = 30
HEAD_CENTER = 90
HEAD_RIGHT = 150
BLOCKED = 30 # closer = obstacle (cm)
```



LESSON 4 – AUTONOMOUS MODES • THE FILE, CHUNK 5

setup(): build only what you need

Both modes drive, so always make the motor – then create only the chosen mode's sensors.

- Parts start as `None` ; build them in `setup()` .
- The `if/else` only builds what the mode needs – the rest stay `None` .
- Sonic mode starts the head looking **straight ahead**.

```
lesson_4_autonomous_modes.py

motor = None
adc = None
servo = None
ultrasonic = None

def setup():
    """Build just the hardware this mode needs."""
    global motor, adc, servo, ultrasonic
    motor = Motor()           # both modes drive
    if DEMO_MODE == "light":
        adc = Adc()           # light sensors
    else:
        servo = Servo()        # head + sensor
        ultrasonic = Ultrasonic()
        servo.setServoPwm("0", HEAD_CENTER)
```



LESSON 4 – AUTONOMOUS MODES • THE FILE, CHUNK 6

light_step(): steer toward light

One pass of Sense → Think → Act for light mode: read both sensors, compare, drive.

- **Sense:** read the left and right light channels.
- **Think:** both bright → straight; one side brighter → turn.
- **Act:** the turn tuples are the same trick as Lesson 2.

```
lesson_4_autonomous_modes.py

def light_step():
    left = adc.recvADC(LEFT_LIGHT)
    right = adc.recvADC(RIGHT_LIGHT)
    # smaller voltage = brighter
    if left < BRIGHT_ENOUGH and right < BRIGHT_ENOUGH:
        motor.setMotorModel(DRIVE_SPEED, DRIVE_SPEED, DRIVE_SPEED, DRIVE_SPEED)
    elif left - right > SIDE_DIFFERENCE:
        motor.setMotorModel(-TURN_SPEED, -TURN_SPEED, TURN_SPEED, TURN_SPEED)
    elif right - left > SIDE_DIFFERENCE:
        motor.setMotorModel(TURN_SPEED, TURN_SPEED, -TURN_SPEED, -TURN_SPEED)
    else:
        motor.setMotorModel(0, 0, 0, 0)
```



LESSON 4 – AUTONOMOUS MODES • THE FILE, CHUNK 7

look(): aim and measure

A small helper – point the head at an angle, wait for it to arrive, then measure.

- Turn the head to `angle`, then read the distance there.
- The `sleep(0.3)` gives the head time to finish moving.
- Reused **three times** in sonic mode – ahead, left, right.

```
lesson_4_autonomous_modes.py

def look(angle):
    """Aim the head, then measure distance (cm)."""
    servo.setServoPwm("0", angle)
    time.sleep(0.3)           # let the head settle
    return ultrasonic.get_distance()
```



LESSON 4 – AUTONOMOUS MODES • THE FILE, CHUNK 8

sonic_step(): look, then turn

If the path ahead is clear, drive. If blocked, stop, look both ways, and turn to open space.

- Clear ahead → drive forward, then `return` early.
- Blocked → stop, measure left and right.
- Turn toward the side with **more room**.

```
lesson_4_autonomous_modes.py

def sonic_step():
    ahead = look(HEAD_CENTER)
    if ahead > BLOCKED:
        motor.setMotorModel(DRIVE_SPEED, DRIVE_SPEED, DRIVE_SPEED, DRIVE_SPEED)
        return # clear road

    motor.setMotorModel(0, 0, 0, 0) # stop and look
    left = look(HEAD_LEFT)
    right = look(HEAD_RIGHT)
    servo.setServoPwm("0", HEAD_CENTER) # re-center the head
    if left > right:
        motor.setMotorModel(-TURN_SPEED, -TURN_SPEED, TURN_SPEED, TURN_SPEED)
    else:
        motor.setMotorModel(TURN_SPEED, TURN_SPEED, -TURN_SPEED, -TURN_SPEED)
    time.sleep(0.5) # turn briefly
```



LESSON 4 – AUTONOMOUS MODES • THE FILE, CHUNK 9

loop(): run the chosen mode

Repeat the right single step forever, with a tiny pause – the Sense → Think → Act loop made real.

- `DEMO_MODE` decides which single step runs each pass.
- The `while True` repeats it many times a second.
- A tiny `sleep(0.05)` keeps the loop from being frantic.

```
lesson_4_autonomous_modes.py

def loop():
    """Repeat the chosen mode's step over and over."""
    while True:
        if DEMO_MODE == "light":
            light_step()
        else:
            sonic_step()
        time.sleep(0.05)           # a tiny pause
```



LESSON 4 – AUTONOMOUS MODES • THE FILE, CHUNK 10

Clean up: destroy()

Stop driving, re-center the head, and close the sensor connection.

- `setMotorModel(0, 0, 0, 0)` stops the wheels.
- Re-center the head and `i2cClose()` the ADC.
- Guard with `is not None` – unused parts were never created.

```
lesson_4_autonomous_modes.py

def destroy():
    """Stop the car and tidy up the hardware."""
    if motor is not None:
        motor.setMotorModel(0, 0, 0, 0)      # stop
    if servo is not None:
        servo.setServoPwm("0", HEAD_CENTER) # face front
    if adc is not None:
        adc.i2cClose()                       # close ADC
```



LESSON 4 – AUTONOMOUS MODES • THE FILE, CHUNK 11

The main block

The familiar wrapper: `try` runs the work, `finally` always cleans up.

- `try` runs `setup()` then `loop()`.
- `except KeyboardInterrupt` catches **Ctrl+C**.
- `finally` guarantees `destroy()` runs no matter what.
- Fourth time seeing this block – you should predict it now.

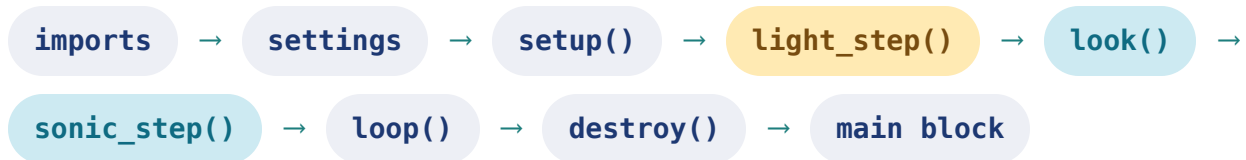
lesson_4_autonomous_modes.py

```
if __name__ == "__main__":  
    try:  
        setup()  
        loop()  
    except KeyboardInterrupt:  
        pass  
    finally:  
        destroy()
```

LESSON 4 – AUTONOMOUS MODES

You built the whole file

One file, two self-driving brains – `loop()` picks which one runs.



The **amber** and **teal** pieces are the two "brains" – `loop()` runs just one of them.



One file, two self-driving modes.



LESSON 4 – AUTONOMOUS MODES

Run it

Set `DEMO_MODE`, then run on the **Pi** – and give it floor space.

- Set `DEMO_MODE` to `"sonic"` or `"light"` first.
- For **light mode**, use a phone flashlight to lead it.
- Press `Ctrl+C` to stop. Use `sudo` for the run command.

Pi terminal

```
cd Smart-Car-Level-3/Code/User  
sudo python3 lesson_4_autonomous_modes.py
```



Clear a driving area first. Watch fingers and table edges.



LESSON 4 – AUTONOMOUS MODES

Try it yourself

Tune the settings, watch how the car's behavior changes, then push further.

- Lower `DRIVE_SPEED` / `TURN_SPEED` if it's too wild.
- Change `BLOCKED` (sonic) – does it stop sooner or later?
- Change `SIDE_DIFFERENCE` (light) – more or less twitchy?

CHALLENGE

React to an obstacle

Beep or **flash an LED** when the car spots an obstacle – reuse the buzzer and LEDs from Lessons 1 & 2.



LESSON 4 – AUTONOMOUS MODES

Common issues

If the driving looks off, it's almost always one of these.



Turns the wrong way (light)

Remember **lower voltage = brighter**; check the left/right wiring.



Spins in circles (sonic)

`BLOCKED` too high, or the head isn't centered. Lower the speeds.



Won't move

Battery low, or `DEMO_MODE` needs sensors that aren't connected.



Head doesn't turn

Servo is on channel `"0"`; give it time – `look()` waits 0.3 s.



Check these first

Most "weird driving" today is a **low battery** or **too-high speed**. Rule those out before anything else.



LESSON 4 – AUTONOMOUS MODES

The car drives itself!

You built the Sense → Think → Act loop and two autonomous modes.

- ✓ Built the **Sense → Think → Act** loop that powers all robots
- ✓ Wrote two modes: follow the light, and dodge obstacles
- ✓ The car now reacts to **light** and **obstacles** on its own

UP NEXT

Lesson 5 Line following

Follow a line on the floor using the infrared sensors – a third kind of "sense and steer".



• COACH ONLY • HIDDEN SLIDE

End-of-class checklist

Don't present this slide. Run through it right after class.

1

Post the after-class recap

Summarise what you covered on the class page so students can catch up.

2

Log absent students

Post anyone who missed class in the **#attendance** Discord channel.

3

Prep for next class

Charge the cars and clear floor space for **Lesson 5 – line following**.

EVERY CLASS

✓ Post a class announcement

✓ Introductions if needed

✓ Note absences in #attendance